

SQL Übungsblatt – Zusatzthemen (Chinook DB, PostgreSQL)

Themen: VIEWS, Systemkatalog, Indizes, Rechteverwaltung

VIEWS

1. **Erstelle einen `SELECT`-Befehl**, der alle Rechnungen mit den zugehörigen Kundendaten anzeigt. Zeige alle Spalten aus `invoice` sowie den `first_name`, `last_name` und `country` aus der Tabelle `customer`.
 2. **Erstelle aus der Abfrage in Aufgabe 1 eine View** mit dem Namen `rechnung_mit_kunde`.
 3. **Erstelle eine View `rock_tracks`**, die alle Tracks mit dem Genre "Rock" enthält. Gib aus: `track.name`, `composer`, `album.title`.
 4. **Verwende die View `rock_tracks`**, um alle Tracks zu ermitteln, deren Name mit "Love" beginnt.
 5. **Erstelle eine View `top_kunden`**, die alle Kunden enthält, deren Rechnungen jeweils über 20 USD liegen.
 6. **Beurteile**, ob über die View `top_kunden` neue Datensätze eingefügt werden können. Begründe deine Antwort anhand der SQL-Standards für änderbare Views.
 7. **Lösche die View `rock_tracks`** wieder aus der Datenbank.
-

Systemkatalog & Indexe

Bearbeitungshinweis für den System Catalog / `Information_schema`: Lassen sie sich zunächst die Attribute der relevanten Tabelle anzeigen (vgl. Vorlesungsunterlage) und überlegen auf welcher Spalte sie das geeignete Selektionskriterium anwenden können.

9. **Finde alle Tabellen, die im Schema `chinook` enthalten sind.**
10. **Gib alle Spaltennamen** der Tabelle `track` über den Systemkatalog aus.
11. **Ermittle welche referentielle Integritätsbedingungen im Schema `,chinook'` existieren.**
12. **Erstelle einen Index** auf der Spalte `last_name` der Tabelle `customer`, um Suchvorgänge zu beschleunigen.
13. **Formuliere den SQL-Befehl**, um diesen Index wieder zu löschen
14. **Erstelle einen eindeutigen Index** auf der Spalte `last_name` der Tabelle `customer`. Warum könnte es sein, dass die Anlage des Index einen Fehler erzeugt?

Rechte & Rollen

15. **Erstelle eine neue Rolle `support_readonly`.**
16. **Gewähre dieser Rolle `SELECT`-Zugriff** auf die Tabelle `customer`.
17. **Lege den Benutzer `tim` an**, vergebe ein Passwort, und ordne ihn der Rolle `support_readonly` zu.
18. **Lösche den User `tim`, die Rolle `support_only` und Widerrufe die `SELECT`-Berechtigung** auf `customer` für die Rolle `support_readonly`.
Beachte dabei die notwendige Reihenfolge der entsprechenden Löschbefehle

Lösungen

Teil 1 – VIEWS

1.

```
SELECT i.*, c.first_name, c.last_name, c.country
FROM invoice i
JOIN customer c ON i.customer_id = c.customer_id;
```

2.

```
CREATE VIEW rechnung_mit_kunde AS
SELECT i.*, c.first_name, c.last_name, c.country
FROM invoice i
JOIN customer c ON i.customer_id = c.customer_id;
```

3.

```
CREATE VIEW rock_tracks AS
SELECT t.name, t.composer, a.title AS album_title
FROM track t
JOIN genre g ON t.genre_id = g.genre_id
JOIN album a ON t.album_id = a.album_id
WHERE g.name = 'Rock';
```

4.

```
SELECT * FROM rock_tracks
WHERE name LIKE 'Love%';
```

5.

```
CREATE VIEW top_kunden AS
SELECT DISTINCT c.*
FROM customer c
JOIN invoice i ON c.customer_id = i.customer_id
WHERE i.total > 20;
```

6.

Nein, ein INSERT über die View top_kunden_status ist nicht erlaubt, weil:

- Sie basiert auf einem JOIN
 - Es existiert eine WHERE-Klausel
 - DISTINCT ist in SELECT enthalten
- Diese Kombination verhindert, dass PostgreSQL die Änderungen sauber auf die Originaltabellen abbilden kann.

7.

```
DROP VIEW rock_tracks;
```

Teil 2 – Systemkatalog & Indexe

9.

```
SELECT *  
FROM Information_Schema.Tables  
WHERE table_schema='chinook';
```

10.

```
SELECT *  
FROM Information_Schema.Columns  
WHERE table_name='track';
```

11.

```
SELECT *  
FROM Information_Schema.referential_constraints  
WHERE constraint_schema = 'chinook';
```

12.

```
CREATE INDEX i_last_name  
ON chinook.customer (last_name);
```

13.

```
DROP INDEX chinook.i_last_name;
```

14.

```
CREATE UNIQUE INDEX i_last_name  
ON chinook.customer (last_name);
```

Ein Nachname ist oftmals nicht eindeutig. Wenn dieser nicht eindeutig ist, dann kann kein Unique Index angelegt werden.

Teil 3 – Rechte & Rollen

15.

```
CREATE ROLE support_readonly;
```

16.

```
GRANT SELECT ON customer TO support_readonly;
```

17.

```
CREATE USER tim WITH PASSWORD 'securePass123';  
GRANT support_readonly TO tim;
```

18.

```
REVOKE SELECT ON customer FROM support_readonly;  
DROP User tim;  
DROP ROLE support_readonly;
```

Als erster Schritt muss das Leserecht auf die Tabelle customer entzogen werden